IPTV Player - Flutter Application

Documentación Técnica, Arquitectura y Guía de Implementación

Flutter Multiplataforma | Dart 3.x | HLS / MPEG-DASH | XMLTV EPG

DESCRIPCIÓN GENERAL

Aplicación IPTV multiplataforma desarrollada en Flutter, optimizada para su ejecución en Android, iOS, Linux, macOS y Web. Permite la reproducción fluida de canales de televisión en directo mediante la importación y parseo en tiempo real de listas de reproducción en formato M3U.

CARACTERÍSTICAS PRINCIPALES

Carga & Parseo Avanzado M3U
Importación vía URL con extracción automática de títulos (`tvg-name`), categorías (`group-title`) y logotipos desde los metadatos.

Filtro & Búsqueda en Tiempo Real
Filtrado dinámico por categorías de lista y motor de búsqueda integrado para consultar por título o URL de canal instantáneamente.

Persistencia de Favoritos
Gestión de catálogo preferido (agregar/quitar) sincronizado con el almacenamiento local mediante `shared_preferences` .

Reproducción de Streams Multi-formato
Integración con `better_player_plus` que proporciona detección autoconfigurada de protocolos HLS (`.m3u8`) y MPEG-DASH (`.mpd`).

Guía Electrónica de Programación (EPG)
Procesamiento XMLTV automático desde etiquetas de cabecera (`url-tvg` , `x-tvg-url`), derivación Xtream (`xmltv.php`) o URL manual.

Controles Avanzados de Player
Controles interactivos con pausa/play, pantalla completa, navegación de canales, detener y **saltos de seek (±10 segundos)** .

Gestión de Audio y Subtítulos
Soporte técnico para la selección dinámica de pistas alternativas de audio y subtítulos mediante las APIs expuestas por el controlador.

Manejo de Errores & Grabación
Intercepción de fallos HTTP (403/404, HTTP no cifrado) con interfaz descriptiva y utilidad para simulación de grabaciones locales en texto.

ARQUITECTURA DEL PROYECTO

El código fuente principal está estructurado de manera modular en dos librerías Dart dentro del directorio `lib/` :

lib/main.dart

- **UI Principal & Estado:** Controladores de entrada de texto, estados de carga y gestión de errores inline.
- **Integración del Player:** Enlace del reproductor `better_player_plus` y lógica de renderizado.
- **Navegación & Búsqueda:** Filtrado dinámico por categorías y barra de búsqueda en vivo.
- **Persistencia Local:** Controladores de lectura/escritura con `shared_preferences` .
- **Controles del Stream:** Lógica de avance/retroceso temporal (seek 10s) y cambio

lib/m3u_parser.dart

- **Clase M3U8Entry:** Modelo de datos optimizado con operadores `==` y `hashCode` por contenido.
- **Función parseM3U():** Parser síncrono que procesa metadatos `#EXTINF` y `#EXTGRP:` .
- **Auxiliares de Atributos:** Métodos `_attribute()` y `_displayName()` para decodificación.
- **Clase M3UPlaylist:** Contenedor agregador de la colección de canales y URL EPG.
- **inferXtreamEpgUri():** Extractor automatizado de parámetros Xtream (`xmltv.php`).

DEPENDENCIAS DEL PROYECTO (PUBSPEC.YAML)

Paquete	Versión	Propósito y Función en la Aplicación
`better_player_plus`	`^1.4.1`	Reproductor de video de alto rendimiento con soporte para HLS, DASH y selección de audio/subtítulos.
`shared_preferences`	`^2.0.5`	Persistencia clave-valor en almacenamiento local para conservar la lista M3U e historial de favoritos.
`xml`	`7.0.1`	Procesamiento y parseo de estructuras de datos XMLTV para la Guía de Programación (EPG).
`http`	`^1.3.0`	Cliente para la realización de peticiones HTTP/HTTPS asíncronas para la carga de listas y guías.
`path_provider`	`^2.0.2`	Acceso simplificado a los directorios del sistema de archivos para exportaciones y archivos temporales.
`flutter_lints`	`^5.0.0`	Conjunto de reglas de análisis estático recomendadas para mantener la calidad y estilo del código Dart.

CORRECCIONES Y MEJORAS APLICADAS (1 AL 5)

1. Igualdad por Contenido en M3U8Entry (`lib/m3u_parser.dart:1-23`)

Se implementaron explícitamente los operadores `==` y `hashCode` comparando las propiedades `url`, `title` y `category`. Esto corrige el comportamiento del método `_toggleFavorite()`, permitiendo identificar correctamente las entradas dentro de conjuntos y mapas sin depender de la identidad de objeto en memoria.

2. Parser M3U Completo y Robusto (`lib/m3u_parser.dart:25-68`)

Reemplazo del generador estático por la función asíncrona `parseM3U()`. Extrae títulos mediante `tvg-name`, detecta agrupaciones con `group-title` y `#EXTGRP:`, resuelve URLs relativas tomando como base el origen de la lista y descarta entradas malformadas sin interrumpir la ejecución.

3. Persistencia de Listas con SharedPreferences (`lib/main.dart:97-165`)

Se añadió un ciclo de almacenamiento automático. La lista parseada se serializa a formato JSON (canales, metadatos y EPG URI) y se guarda localmente. Al iniciar la aplicación, `_loadSavedPlaylist()` restaura el catálogo omitiendo la re-descarga obligatoria.

4. Robustez en la Carga HTTP (`lib/main.dart:47-106`)

El método `_loadM3U` valida los esquemas URI (HTTP/HTTPS) previo a la solicitud, añade un límite de tiempo de espera (timeout) de 20 segundos y encapsula la operación en bloques `try/catch/finally` para asegurar la limpieza de los indicadores de carga y notificar errores claros.

5. Optimización del Reproductor BetterPlayer (`lib/main.dart:108-139`)

Detección dinámica de tipo de contenido: asignación de `BetterPlayerVideoFormat.hls` para URLs `.m3u8` y `dash` para `.mpd`. Se habilitan los selectores de audio y subtítulos en la configuración del controlador y se añade `autoPlay: true`.

CORRECCIONES Y MEJORAS APLICADAS (6 AL 10)

6. Widget PlayerErrorView

Componente de interfaz aislado (`lib/main.dart:359-417`) diseñado para capturar fallos de reproducción. Renderiza iconos contextuales, explicaciones de error y un botón "Reintentar" respetando el `AspectRatio` contenedor.

7. Búsqueda en Tiempo Real

Implementación del estado `_searchQuery` en la barra superior. Filtra en directo la lista activa comparando el texto ingresado con los títulos y URLs de los canales mediante `setState`.

8. Botones Seek (±10 Segundos)

Adición de controles temporales mediante `_seekRelative(int seconds)`. Calcula la posición actual del controlador de video y ejecuta `controller.seekTo()` de forma segura.

9. Gestión de Ciclo de Vida

Liberación explícita de recursos dentro del método `dispose()` en `lib/main.dart`, cerrando `_urlController`, `_searchController` y el reproductor para prevenir fugas de memoria.

10. Detección Inteligente de EPG Xtream Codes (`lib/m3u_parser.dart:34-68`)

La función `inferXtreamEpgUri()` analiza listas provenientes de servidores Xtream Codes (ej. `572369.solanaflix.com`), transformando dinámicamente rutas `get.php` hacia `xmltv.php` conservando los parámetros de autenticación `username` y `password`.

GUÍA DE EJECUCIÓN DEL PROYECTO

Requisitos Previos: SDK de Flutter instalado (compatible con Dart 3.x) y dispositivo/emulador configurado.
Comandos de Inicialización:

```
flutter pub get
flutter run
```

PANTALLA EPG & ESPECIFICACIONES DEL REPRODUCTOR

La interfaz de la Guía EPG permite visualizar el archivo XMLTV descargado (soporta respuestas estándar y comprimidas mediante gzip). Las asociaciones se realizan contrastando el atributo `tvg-id` del canal M3U con el atributo `channel` de la guía XMLTV. El reproductor integrado ofrece los siguientes controles de transmisión:

- **Canal Anterior / Siguiente:** Desplazamiento secuencial en el catálogo filtrado.
- **Pausa / Reanudar / Detener:** Gestión del estado de reproducción del stream.
- **Saltos Temporales (⏪ Rebobinar / ⏩ Avanzar 10s):** Desplazamiento relativo en streams HLS/VOD.
- **Pantalla Completa:** Modos de visualización inmersivos.

Advertencias & Consideraciones Conocidas:

- **Análisis Estático:** `dart analyze` reporta **0 issues** (No issues found ✓).
- **Persistencia:** Los datos persisten mediante `shared_preferences` a nivel de dispositivo.
- **Conectividad:** Requiere acceso a red activo; no cuenta con modo offline para retransmisión.
- **Identificadores EPG:** Es indispensable la coincidencia entre `tvg-id` y el atributo `channel` del XMLTV para renderizar la programación correctamente.